PC CRM

Comprehensive Developer Architecture & Implementation Guide

The Complete Full-Stack Engineering Manual: Deep Architectural Mechanics, Directory & File-by-File Breakdown, Schema Topology, Dynamic Form & View Systems, Sandboxed Scripting Engine, and Step-by-Step Scratch Blueprint.

System:	PC CRM (Self-Hosted Enterprise CRM Platform)
Frontend Tech:	Vue 3 (Composition API), Vite, Tailwind CSS, Frappe UI, Pinia Stores
Backend Tech:	Frappe Framework, Python 3.14, MariaDB 11.8, Redis 8 (Queue/Cache/Socket.io)
Target Audience:	Software Engineers, Core Contributors, System Architects, DevOps Engineers
Repository:	Forked from Frappe CRM (Autonomous, Rebranded & Self-Sovereign)
Specification Date:	October 2026 — Local Production Ready

MANUAL PURPOSE:

This guide is written specifically from an engineering perspective. It provides the deep structural understanding required not only to maintain, customize, and extend PC CRM, but to reconstruct its entire architecture from absolute scratch. Every architectural pattern, routing behavior, database abstraction, frontend component, and API contract is explicitly documented.

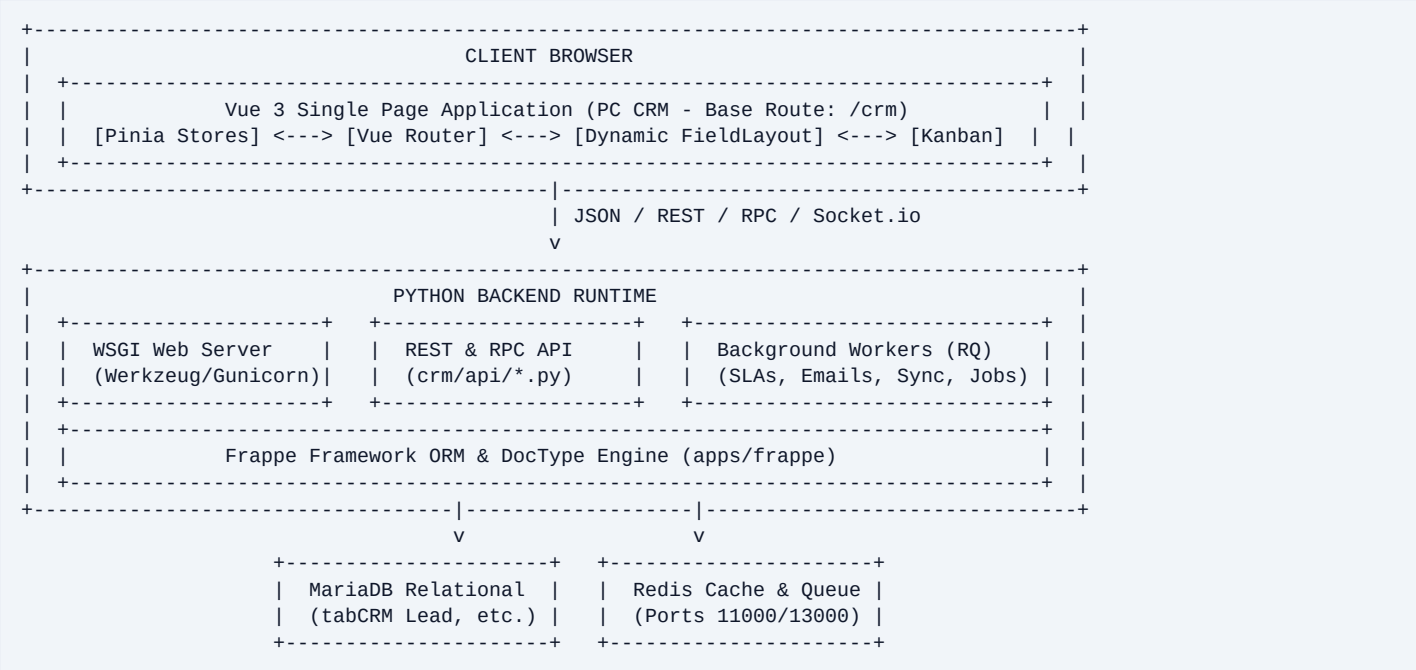
Document Structure & Learning Roadmap

Chapter	Title & Core Concepts Covered	Key Focus
Chapter 1	**Full-Stack Architecture & Runtime Topology** Dual-mode routing, HTTP request flow, session lifecycle, boot injection, Redis/MariaDB architecture.	System Foundations
Chapter 2	**Backend Architecture & Core Modules** Hooks manifest, complete DocType catalog, REST/RPC API directory, domain enrichment, lead syncing, telephony & ERPNext integrations.	Python Backend & DB
Chapter 3	**Frontend Architecture & Component Deep Dive** Vue 3 SPA boot, Pinia stores, router guards, dynamic FieldLayout engine, custom Controls, Kanban boards, Activity timeline, and Modals.	Vue 3 Frontend
Chapter 4	**The CRM Form Scripting Engine** Low-code runtime extensibility, client-side script compilation (new Function), DocProxy reactivity, lifecycle hooks, setFieldProperty, and formDialog API.	Dynamic Scripting
Chapter 5	**How to Code This System From Scratch** Step-by-step master engineering blueprint: DB modeling, REST layer, metadata engine, schema-driven form builder, boards, feeds, and deployment.	Scratch Implementation
Chapter 6	**Exhaustive File-by-File Directory Catalog** Comprehensive index of every single file in the repository with path, primary exports, responsibilities, and architectural purpose.	Complete Reference

Chapter 1: Full-Stack Architecture & Runtime Topology

1.1 The High-Level Architecture

PC CRM is a modern, reactive Customer Relationship Management platform built with a decoupled Single Page Application (SPA) frontend sitting on top of an enterprise Python backend runtime (Frappe Framework). It does not use traditional server-rendered templates for the CRM user interface; instead, the entire CRM workspace is an independent Vue 3 application compiled with Vite.



1.2 Dual-Mode Routing Mechanics

A foundational architectural feature of PC CRM is its **Dual-Mode Routing Topology**. The system simultaneously supports two distinct interfaces within a single server instance:

1. The Standalone CRM SPA (Mounted at /crm): Built for day-to-day sales agents, marketing managers, and executives. It provides a modern, fast, responsive interface with custom pipelines, Kanban boards, and an omnichannel activity hub. It is served by crm/www/crm.py and crm/www/crm.html, and managed client-side by Vue Router configured with createWebHistory('/crm').

2. The Frappe Desk Interface (Mounted at /desk or /app): Built for system administrators, DevOps, and technical architects. It provides low-level schema management (DocType builder), role-based permissions management, database migration runners, automated workflow triggers, and raw database console access.

1.3 The Bootstrapping Lifecycle (How the Browser Boots)

When a user navigates to http://localhost:8000/crm, the following sequence executes:

Step 1: Python Route Handling (crm/www/crm.py): Frappe's website router intercepts the request matching website_route_rules in crm/hooks.py. The get_context() function runs in Python. It validates user authentication, executes check_app_permission(), and builds the initial boot dictionary via get_boot().

Step 2: Server-Side Boot Injection: crm/www/crm.html outputs a lightweight HTML skeleton. Inside the <head>, it injects the complete boot payload into the global window.frappe JavaScript namespace. This payload contains current user session info, user roles, system timezone, CSRF tokens, currency settings, installed apps, and system defaults. Because this data is pre-injected into HTML, the frontend does not need to make 5 separate HTTP calls just to initialize.

Step 3: Vue 3 Hydration & Store Mount (frontend/src/main.js): The browser downloads the compiled Vite bundle (main.js). Vue mounts the root component App.vue, registers Pinia state management stores, initializes frappe-ui components, attaches global error handlers, and invokes the router.

Step 4: Client-Side Route Resolution (frontend/src/router.js): The router evaluates the path. When landing on /crm (route name 'Home'), the beforeEach guard checks the user's default saved view from viewsStore(). If no view is preferred, it redirects immediately to /crm/leads (Leads view).

1.4 Authentication, Session Management & Security

PC CRM enforces a dual authentication model:

1. Cookie-Based Session Auth: When logging in via the web form (/api/method/login), Frappe generates an encrypted session record in MariaDB (tabSessions) and sets an HTTP-only cookie sid=<session_id> with SameSite=Lax. Subsequent browser requests automatically carry this cookie.

2. API Key & Secret Token Auth: For external API integrations, mobile apps, or cURL requests, the server validates HTTP headers formatted as: Authorization: token <api_key>:<api_secret>. Frappe's validate_auth_via_api_keys() in frappe/auth.py decodes the key, fetches the corresponding user from tabUser, verifies the encrypted secret with HMAC constant-time digest comparison, and initializes the user session for that request.

Chapter 2: Backend Architecture & Core Modules

2.1 The hooks.py Manifest — The Central Wiring Harness

In Frappe architecture, an app declares its integration points with the framework inside crm/hooks.py. This file is the single source of truth for routing, event listeners, background tasks, and UI extensions. Here are the key hook definitions:

Hook Name	Code Setting in crm/hooks.py	Architectural Purpose
`app_title`	"PC CRM"	The application brand name displayed in desktop and page headers.
`website_route_rules`	[{"from_route": "/crm/", "to_route": "crm"}]	Directs all browser traffic under /crm/* to the single-page HTML loader crm.html.
`add_to_apps_screen`	[{"name": "crm", "title": "PC CRM", "route": "/crm"}]	Registers the CRM icon on Frappe Desk's application switcher rail.
`doctype_js`	{"CRM Lead": "public/js/domain_enrichment.js"}	Injects custom client scripts into standard Desk views for domain enrichment.
`doc_events`	{"Contact": {"after_insert": "crm.api.contact.after_insert"}}	Triggers backend business logic whenever specific DocTypes are created or updated.
`scheduler_events`	{"cron": {"*/5 * * * *": ["crm.fcrm.doctype...check_sla"]}}	Runs background cron jobs for SLA tracking, email sync, and assignment rules.

2.2 Complete DocType Catalog & Schema Topology

A **DocType** is Frappe's declarative model schema. It defines both the database table schema (in MariaDB) and the field behaviors, validations, and links. All CRM DocTypes reside in crm/fcrm/doctype/:

DocType Name	Table Name (MariaDB)	Key Fields & Schema Responsibilities
CRM Lead	`tabCRM Lead`	**Fields:** first_name, last_name, email, mobile_no, status, lead_owner, organization, annual_revenue, sla_status, converted. **Role:** Central prospect record. Tracks marketing source, communication logs, and conversion to Deal.
CRM Deal	`tabCRM Deal`	**Fields:** deal_name, deal_owner, status, organization, expected_close_date, annual_revenue, deal_value, probability. **Role:** Active sales opportunity progressing through customizable pipeline stages.
CRM Organization	`tabCRM Organization`	**Fields:** organization_name, website, domain, industry, no_of_employees, annual_revenue, territory. **Role:** Corporate account entity. Parent to multiple contacts, leads, and deals.
Contact	`tabContact`	**Fields:** first_name, last_name, email_ids, phone_nos, links (Dynamic Link). **Role:** Individual person entity linked to Organizations, Deals, and Leads.
CRM Call Log	`tabCRM Call Log`	**Fields:** from, to, call_type (Inbound/Outbound), duration, recording_url, telephony_agent. **Role:** Telephony integration tracking phone interactions and playback audio.
CRM Task	`tabCRM Task`	**Fields:** title, assigned_to, due_date, priority, status, reference_doctype, reference_docname. **Role:** Actionable to-do items linked to CRM records with notifications.
FCRM Note	`tabFCRM Note`	**Fields:** title, content (Markdown/HTML), assigned_to, reference_doctype, reference_docname. **Role:** Internal collaboration notes visible in the document activity timeline.
CRM Service Level Agreement	`tabCRM Service Level Agreement`	**Fields:** entity_type, holiday_list, service_level_priorities (Child Table), service_days. **Role:** SLA policies calculating first-response and resolution target deadlines.

CRM Form Script	`tabCRM Form Script`	**Fields:** doc_type, view, script (Long Text), enabled. **Role:** Stores custom JavaScript evaluated in the browser for low-code form automation.
CRM Fields Layout	`tabCRM Fields Layout`	**Fields:** dt (DocType), layout (JSON structure of tabs, sections, and fields). **Role:** Configurable form layouts dynamically rendered by the Vue FieldLayout engine.

2.3 The Backend API Engine (crm/api/)

All endpoints exposed to the frontend are whitelisted Python functions decorated with @frappe.whitelist() in crm/api/. They can be invoked via HTTP POST/GET to /api/method/crm.api.<module>.<function>:

Module File	Key Whitelisted Endpoints	Functionality & Query Mechanics
`activities.py`	get_activities(doctype, name) add_comment(), delete_activity()	Combines Comments, Communications (Emails), Call Logs, Tasks, Notes, and Version audit diffs into a unified, reverse-chronological activity feed for the detail view.
`doc.py`	get_doc_info() convert_lead_to_deal() get_sidebar_summary()	Handles document lifecycle transitions. convert_lead_to_deal creates an Organization, Contact, and Deal from a Lead atomically inside a database transaction.
`views.py`	get_views(doctype) save_view() delete_view()	Manages user-customized views (List, Kanban, Group-by), persisting filter conditions, sort orders, visible columns, and default view preferences in MariaDB.
`assignment_rule.py`	apply_assignment_rule() get_available_agents()	Executes automated round-robin or criteria-based lead distribution across sales agents based on active working hours and territory rules.
`dashboard.py`	get_dashboard_data() get_chart_data()	Aggregates real-time CRM analytics: total pipeline value, conversion funnels, win/loss ratios, lead velocity, and agent performance charts.
`whatsapp.py`	send_whatsapp_message() get_templates()	Integrates WhatsApp messaging API for direct conversational outreach from the Lead/Deal activity timeline.

2.4 Domain Enrichment & Automation Subsystems

1. Domain Enrichment (crm/domain_enrichment/): When a lead enters the system with an email like john@acme.com, the enrichment worker extracts the domain (acme.com). It matches existing CRM Organization records by domain or executes an automated enrichment rule (skipping public providers like gmail.com). If a corporate domain matches, it automatically links the lead to the company and enriches industry, logo, and employee size.

2. Lead Syncing (crm/lead_syncing/): Webhook receivers designed for Facebook Lead Ads. Ingests raw JSON payloads, maps dynamic form questions to Lead fields, and creates lead records while logging any failed attempts in tabFailed Lead Sync Log for automated retry.

3. Telephony & ERPNext Integrations (crm/integrations/): Webhooks for Twilio and Exotel phone systems. When a call completes, the telephony provider pushes duration, agent ID, and audio recording MP3 URLs into CRM Call Log. For ERPNext, it automatically syncs Item Master catalogs, Quotations, and Sales Orders directly into the CRM deal view.

Chapter 3: Frontend Architecture & Component Deep Dive

3.1 The Frontend Directory Structure

The frontend is located in frontend/src/. It is organized into specialized layers following Vue 3 best practices:

Directory	Responsibility	Key Files / Modules
`stores/`	Pinia reactive global state containers.	meta.js, views.js, users.js, session.js, settings.js
`composables/`	Reusable Vue composition functions & state hooks.	document.js, modals.js, telephony.js, useUndoHistory.js
`components/FieldLayout/`	Dynamic schema-driven form layout renderer.	FieldLayout.vue, Field.vue, Section.vue, Column.vue
`components/Controls/`	Custom UI input controls for all fieldtypes.	Link.vue, Grid.vue, RatingInput.vue, DurationInput.vue
`components/ListViews/`	Tabular lists, custom columns, bulk actions.	LeadsListView.vue, DealsListView.vue, ListRows.vue
`components/Kanban/`	Drag-and-drop visual pipeline board.	Kanban.vue, KanbanColumn.vue, KanbanCard.vue
`components/Activities/`	Omnichannel activity feed & timeline hub.	Activities.vue, CommentArea.vue, EmailArea.vue, CallArea.vue
`components/Modals/`	Application-wide dialogs & quick-entry forms.	FieldLayoutDialog.vue, GlobalModals.vue, ConvertToDealModal.vue
`data/`	Scripting engine & document data contracts.	document.js (useDocument), script.js (getScript)
`utils/`	Pure utility functions & expression evaluators.	fieldTransforms.js, expressions.js, renderFieldLayoutDialog.js

3.2 State Management: Pinia Stores Deep Dive

1. metaStore (frontend/src/stores/meta.js): The central metadata cache. When the UI needs to render any DocType, getMeta(doctype) fetches the schema from the server once and caches it in memory. It exposes getFields(), field formatters, link targets, and title fields, eliminating redundant schema API calls.

2. viewsStore (frontend/src/stores/views.js): Manages active views. It maintains the list of standard views (List, Kanban, Group-by) and custom user views. It tracks active filter arrays, sorting criteria, column widths, and visible fields. When a user changes a filter or reorders columns, viewsStore handles state and synchronization with MariaDB.

3. sessionStore & usersStore (stores/session.js, stores/users.js): sessionStore maintains user login state, full name, email, and logout actions. usersStore fetches and caches user profiles, avatars, roles, and provides helper checks like isAdmin() and isCrmUser().

3.3 The Dynamic Schema-Driven Form Engine (components/FieldLayout/)

Instead of hardcoding HTML forms for Leads, Deals, and Organizations, PC CRM implements a **generic, schema-driven layout engine**. This allows administrators to customize form fields, reorder sections, and add custom tabs through Desk without writing any frontend code.

The layout hierarchy operates as follows:

<FieldLayout /> → Reads crm_fields_layout JSON. Organizes fields into Tabs.
<Section /> → Renders collapsible sections with titles and descriptions.
<Column /> → Implements flexible multi-column grid layouts (1 to 4 columns).
<Field /> → Evaluates depends_on expressions via utils/expressions.js. If visible, resolves the fieldtype (Data, Link, Check, Select, Date, Table) and renders the appropriate control from components/Controls/.

3.4 The Omnichannel Activity Timeline (components/Activities/)

The right-hand side of any Lead or Deal detail view is powered by Activities.vue. This is an omnichannel communication hub that merges:

- **Comments:** Internal team messages with user mentions and formatting.
- **Emails:** Full incoming and outgoing email threads via IMAP/SMTP with rich text and attachments.
- **Call Recordings:** Embedded HTML5 audio player for listening to telephony recordings.
- **Tasks & Notes:** Actionable to-do checkboxes and pinned documentation notes.
- **Audit Diff Log:** Automatic tracking of field changes (e.g., status changed from 'New' to 'Qualified').

Chapter 4: The CRM Form Scripting Engine (Low-Code Extensibility)

4.1 Why Form Scripting Exists

In modern enterprise CRMs, business logic evolves faster than deployment cycles. PC CRM includes an in-browser **Form Scripting Engine** that allows developers and administrators to write plain JavaScript (ES Classes) stored in the database (CRM Form Script) that attach directly to forms on the fly — **with zero build step and zero server redeployments**.

4.2 The Evaluation Pipeline (frontend/src/data/script.js)

How a script stored as a string in MariaDB executes safely inside the Vue 3 component tree:

```
[1. Fetch Script]  MariaDB (tabCRM Form Script) ---> frontend/src/data/script.js: getScript(doctype)
                           |
[2. Evaluate Class]        |---> Compiled via: new Function('frappe', 'call', ..., scriptContent)
                           |---> Returns the evaluated class (e.g. class CRMLead)
                           v
[3. Instantiate & Wrap]    |---> Instantiated: scriptInstance = new ScriptClass()
                           |---> Document Proxy: this.doc wrapped with Proxy via createDocProxy()
                           |---> Helpers Injected: this.setFieldProperty, this.formDialog, this.makeCall
                           v
[4. Lifecycle Binding]     |---> frontend/src/data/document.js (useDocument)
                                 Hooks wired: onLoad, onRender, onValidate, onSave, onError, field changes
```

4.3 The Class Contract & Lifecycle Hooks

A form script must define an ES Class whose name matches the DocType name with spaces stripped (e.g., CRM Lead → class CRMLead):

```
class CRMLead {
  async onLoad() {
    // Fires once when document loads from server
    if (this.doc.annual_revenue > 1000000) {
      this.setFieldProperty('priority', 'options', 'High\nCritical');
    }
  }

  status() {
    // Fires whenever the 'status' field changes
    if (this.value === 'Lost') {
      this.setFieldProperty('lost_reason', 'mandatory', true);
      this.setFieldProperty('lost_reason', 'hidden', false);
    } else {
      this.setFieldProperty('lost_reason', 'mandatory', false);
      this.setFieldProperty('lost_reason', 'hidden', true);
    }
  }

  async onValidate() {
    // Before save: validate and throw to block save
    if (this.doc.status === 'Lost' && !this.doc.lost_reason) {
      throw new Error('Lost Reason is mandatory when status is Lost!');
    }
  }

  async onSave() {
    // After successful save
    frappe.toast({ message: 'Lead updated successfully', indicator: 'green' });
  }
}
```

4.4 The setFieldProperty API Specification

Inside any hook, this.setFieldProperty(target, property, value [, rowName]) dynamically alters the rendering state of any field, section, tab, or child table column:

Property	Type	Description & Effect on Field
`hidden`	Boolean	Hides or reveals the field, section, tab, or child table column.
`read_only`	Boolean	Toggles read-only state. Prevents user input while maintaining display.
`mandatory`	Boolean	Enforces required field validation before save completes.
`options`	String/Array	Dynamically changes dropdown Select options (newline or comma-separated).
`label`	String	Dynamically updates the field label text.
`placeholder`	String	Updates the HTML input placeholder.
`description`	String	Displays helper instructions beneath the input control.

Chapter 5: How to Code This System From Scratch

5.1 Master Implementation Blueprint

To rebuild PC CRM from absolute scratch, follow this phased engineering sequence. Each phase establishes the dependency layer required by the subsequent phase.

Phase 1: Environment & Project Scaffolding

• Initialize Bench: Install Frappe Bench CLI (bench init frappe-bench).

• Generate App Skeleton: Run 'bench new-app crm'. This creates the Python module structure, hooks.py, and setup.py.

• Scaffold Frontend: In apps/crm, run 'npx create-vite frontend --template vue'. Install tailwindcss, @tailwindcss/forms, and frappe-ui.

Phase 2: Relational Data Models & DocTypes

• Create Core DocTypes: Define JSON schemas for CRM Lead, CRM Deal, CRM Organization, Contact, CRM Task, and FCRM Note.

• Configure Relations: Set up Dynamic Link tables connecting tasks, notes, and call logs to any CRM record.

• Run Migrations: Execute 'bench --site crm.localhost migrate' to generate the MariaDB physical tables and indexes.

Phase 3: Backend REST & RPC Services

• Unified Activity Aggregator (activities.py): Implement SQL unions combining comments, emails, and audit logs.

• Conversion Logic (doc.py): Write the lead-to-deal transition script that maps lead data into a deal and customer record inside a single db.commit() transaction.

• View Persistence (views.py): Build endpoints to save and load user view preferences from tabCRM View Settings.

Phase 4: Frontend Foundation & State Architecture

• Router Configuration: Configure Vue Router with createWebHistory('/crm') and register beforeEach auth guards.

• Pinia Stores: Implement meta.js for caching schema metadata, session.js for user context, and views.js for filter state.

• App Layout Shell: Build DesktopLayout.vue with responsive CollapsibleSidebar, GlobalSearch, and AppHeader.

Phase 5: The Dynamic Schema-Driven Form Engine

• FieldLayout Component: Build the recursive layout renderer (FieldLayout -> Section -> Column -> Field).

• Expressions Engine: Implement expressions.js with evaluateDependsOnValue() to support dynamic field visibility conditions.

• Custom Controls: Implement specialized inputs (Link autocomplete, Date picker, Rich text editor, Duration, Rating).

Phase 6: Boards, Lists & Activity Feeds

• Kanban Board: Build Kanban.vue with HTML5 Drag-and-Drop. Wire drop events to status update API calls.

• Virtualized Data Table: Implement ListRows.vue supporting column sorting, multi-row bulk actions, and pagination.

• Timeline Hub: Implement Activities.vue with real-time comments, email composers, and audio playback.

Phase 7: Sandboxed Scripting Engine

• Class Evaluator: Implement script.js using 'new Function' to compile user-submitted script strings.

• DocProxy: Wrap reactive document objects in a Proxy to intercept read/write operations and trigger change hooks.

• Dialog Helper: Implement renderFieldLayoutDialog.js to expose programmatic form dialogs (formDialog).

5.2 Key Code Example: Creating a Lead Programmatically

Backend Python implementation for creating leads via API or internal service:

```
import frappe

@frappe.whitelist()
def create_lead(first_name, email, organization_name=None, mobile_no=None, lead_owner=None):
    lead = frappe.new_doc("CRM Lead")
    lead.first_name = first_name
    lead.email = email
    lead.mobile_no = mobile_no
    lead.lead_owner = lead_owner or frappe.session.user
    lead.status = "New"

    if organization_name:
        # Link or create organization
        if not frappe.db.exists("CRM Organization", {"organization_name": organization_name}):
            org = frappe.new_doc("CRM Organization")
            org.organization_name = organization_name
            org.insert(ignore_permissions=True)
            lead.organization = org.name
        else:
            lead.organization = frappe.db.get_value("CRM Organization", {"organization_name": organization_name},
"name")

    lead.insert(ignore_permissions=False)
    frappe.db.commit()
    return lead.as_dict()
```

Chapter 6: Exhaustive File-by-File Directory Catalog

6.1 Backend Python Modules (crm/)

Detailed breakdown of key backend files in the crm/ application:

File Path	Primary Exports / Classes	Architectural Responsibility
`crm/hooks.py`	Config dictionary	App manifest, website routing, desk rails, doc events, and cron schedules.
`crm/install.py`	after_install(), setup()	Post-installation seeder: creates default statuses, SLA policies, and admin settings.
`crm/www/crm.py`	get_context(), get_boot()	Entrypoint for /crm route. Validates auth and builds global boot payload.
`crm/www/crm.html`	HTML Template	Serves HTML skeleton and injects window.frappe.boot JSON payload.
`crm/api/activities.py`	get_activities()	SQL union combining comments, emails, tasks, calls, and version diffs.
`crm/api/doc.py`	convert_lead_to_deal()	Atomic transaction converting a Lead to Organization, Contact, and Deal.
`crm/api/views.py`	get_views(), save_view()	CRUD API for user-customized Kanban and table list view configurations.
`crm/api/assignment_rule.py`	apply_assignment_rule()	Automated round-robin distribution of incoming leads to sales agents.
`crm/api/dashboard.py`	get_dashboard_data()	Aggregates pipeline analytics, deal velocity, and conversion ratios.
`crm/api/whatsapp.py`	send_whatsapp_message()	Sends conversational WhatsApp messages and template notifications.

6.2 Frontend Architecture (frontend/src/)

Detailed breakdown of key frontend stores, composables, layouts, and components:

File Path	Component / Export	Architectural Responsibility
`src/main.js`	Vue App Bootstrapper	Initializes Vue 3, Pinia, Vue Router, Frappe UI, and mounts root App.vue.
`src/router.js`	Vue Router Instance	Configures /crm routes, auth navigation guards, and dynamic view routing.
`src/stores/meta.js`	metaStore	Global in-memory cache for DocType schemas, field types, and link formatters.
`src/stores/views.js`	viewsStore	Manages active table/kanban view filters, column selections, and sort orders.
`src/stores/session.js`	sessionStore	Manages active user credentials, session state, and logout actions.
`src/data/script.js`	getScript()	Fetches CRM Form Scripts from server, evaluates class via new Function, injects helpers.
`src/data/document.js`	useDocument()	Manages document reactivity, dirty state tracking, auto-save, and script hook binding.
`src/components/FieldLayout/FieldLayout.vue`	FieldLayout Component	Master schema layout container rendering tabs, sections, and responsive columns.
`src/components/FieldLayout/Field.vue`	Field Component	Evaluates depends_on expressions and renders appropriate Controls input.
`src/components/Controls/Link.vue`	Link Component	Custom searchable autocomplete dropdown for relational foreign key links.

`src/components/Controls/Grid.vue`	Grid Component	Editable child table component for line items, products, and contact lists.
`src/components/Kanban/Kanban.vue`	Kanban Component	HTML5 drag-and-drop pipeline board with card count and value totals.
`src/components/Activities/Activities.vue`	Activities Component	Unified activity feed merging emails, comments, audio calls, and audit logs.
`src/components/Modals/FieldLayoutDialog.vue`	FieldLayoutDialog Component	Programmatic modal dialog powered by the standalone FieldLayout engine.

ENGINEERING SUMMARY & NEXT STEPS:

You now have the complete architectural specification and engineering reference for PC CRM. With this knowledge, you can independently modify schemas, build custom frontend interfaces, write advanced form scripts, or code an entire enterprise CRM platform from scratch.